49. Глобальные переменные

Мы уже знаем, что переменные, объявленные внутри блока, называются **локальными**. Они имеют локальную область видимости (используется только внутри блока, в котором объявлены) и автоматическую продолжительность жизни (создается в точке определения и уничтожается в конце блока).

Глобальными называются переменные, которые объявлены вне блока. Они имеют **статическую продолжительность жизни**, т.е. создаются при запуске программы и уничтожаются при её завершении. Глобальные переменные имеют **глобальную область видимости** (или **«файловую область видимости»**), т.е. их можно использовать в любом месте файла, после их объявления.

Оглавление:

1. Определение глобальных переменных
2. Ключевые слова static и extern
3. Предварительные объявления переменных с использованием extern
4. Связи функций
5. Файловая область видимости vs. Глобальная область видимости
6. Глобальные символьные константы
7. Предостережение о (неконстантных) глобальных переменных
8. Заключение
9. Тест

Определение глобальных переменных

Обычно глобальные переменные объявляют в верхней части кода, ниже **директив #include**, но выше любого другого кода. Например:

```
#include <iostream>

// Переменные g_x и g_y объявлены вне блока, являются глобальными переменными
int g_x{ 7 }; // по умолчанию g_x
const int g_z{3}; // константная глобальная переменная g_z

void doSomething();
{
    // Глобальные переменные можно использовать в любом месте программы
    g_x = 4;
    std::cout << g_y << '\n';
}

int main()
{
    doSomething();

    // Глобальные переменные можно использовать в любом месте программы
    g_x = 7;
    std::cout << g_y << '\n';

    return 0;
}
```

Подобно тому, как переменные во внутреннем блоке скрывают переменные с теми же именами во внешнем блоке, локальные переменные скрывают глобальные переменные с одинаковыми именами внутри блока, в котором они определены. Однако с помощью **оператора разрешения области видимости** (::), компилятору можно сообщить, какую версию переменной вы хотите использовать: глобальную или локальную. Например:

```
#include <iostream>

int value(){; // глобальная переменная

int main()
{
    int value = 5; // эта переменная (локальная) скрывает значение глобальной переменной
    value = 1; // присваивание локальной переменной value (не глобальной)
    ::value = 2; // присваивание глобальной переменной value (не локальной)

    std::cout << "Global value: " << value << '\n';
    std::cout << "Local value: " << value << '\n';
    return 0;
}
```

Результат выполнения программы:

```
Global value: 3
Local value: 5
```

Использовать одинаковые имена для локальных и глобальных переменных — это прямой путь к проблемам и ошибкам, поэтому подобное делать не рекомендуется. Многие разработчики добавляют к глобальным переменным префикс `g_` (**«g»** от англ. «global»). Таким образом, можно убить сразу двух зайцев: определить глобальные переменные и избежать **конфликтов имен** с локальными переменными.

Ключевые слова static и extern

В дополнение к области видимости и продолжительности жизни, переменные имеют еще одно свойство — связь. **Связь переменной** определяет, относится ли несколько упоминаний одного идентификатора к одной и той же переменной или нет.

Переменная без связей — это переменная с локальной областью видимости, которая относится только в блоке, в котором она определена. Это обычные локальные переменные. Две переменные с одинаковыми именами, но определенными в разных функциях, не имеют никакой связи — каждая из них считается независимой единицей.

Переменная, имеющая внутреннюю связь, называется **внутренней переменной** (или **«статической переменной»**). Она может использоваться в любом месте файла, в котором определена, но не относится к чему-либо вне этого файла.

Переменная, имеющая внешнюю связь, называется **внешней переменной**. Она может использоваться как в файле, в котором определена, так и в других файлах.

Если вы хотите сделать глобальную переменную внутренней (которую можно использовать только внутри одного файла) — используйте **ключевое слово static**:

```
#include <iostream>

static int g_x{ 2 }; // g_x - это статическая глобальная переменная, которую можно использовать только внутри этого файла

int main()
{
    return 0;
}
```

Аналогично, если вы хотите сделать глобальную переменную внешней (которую можно использовать в любом файле программы) — используйте **ключевое слово extern**:

```
#include <iostream>

extern double g_y(9.9); // g_y - это внешняя глобальная переменная и её можно использовать и в других файлах программы

int main()
{
    return 0;
}
```

По умолчанию, неконстантные переменные, объявленные вне блока, считаются внешними. Однако константные переменные, объявленные вне блока, считаются внутренними.

Предварительные объявления переменных с использованием extern

Мы уже знаем, что для использования функций, которые определены в другом файле, нужно применить **предварительные объявления**.

Аналогично, чтобы использовать внешнюю глобальную переменную, которая была объявлена в другом файле, нужно записать предварительное объявление переменной с использованием ключевого слова `extern` (без инициализирующего значения). Например:

global.cpp:

```
#include <iostream>

int g_x{ 2 }; // внешняя глобальная переменная
int g_y(3); // константная глобальная переменная имеет внешнюю связь по умолчанию
// g_z не может использовать в файлах после этого файла
```

main.cpp:

```
#include <iostream>

extern int g_x; // предварительное объявление g_x. Теперь g_x можно использовать в любом месте этого файла

int main()
{
    extern int g_x; // предварительное объявление g_x. Теперь g_x можно использовать только внутри main()

    g_x = 5;
    std::cout << g_x; // выведет значение 5

    return 0;
}
```

Если предварительное объявление находится вне блока, то оно применяется ко всему файлу. Если же внутри блока, то оно применяется только к нему.

Если переменная объявлена с помощью ключевого слова `static`, то получить доступ к ней с помощью предварительного объявления не получится. Например:

constants.cpp:

```
static const double g_gravity(9.8);
```

main.cpp:

```
#include <iostream>

extern const double g_gravity; // не работает g_gravity = constants.cpp, так как g_gravity является внутренней переменной

int main()
{
    std::cout << g_gravity; // выведет ошибку компилятора, так как переменная g_gravity не была определена для использования в этом файле
    return 0;
}
```

Обратите внимание, если вы хотите объявить неинициализированную неконстантную глобальную переменную, то не используйте ключевое слово `extern`, иначе C++ будет думать, что вы пытаетесь записать предварительное объявление.

Связи функций

Функции имеют такие же свойства связи, что и переменные. По умолчанию они имеют внешнюю связь, которую можно сменить на внутреннюю с помощью ключевого слова `static`:

```
// Функция add() определена в этом файле и может использоваться в других файлах
// Обратите внимание, что в этом случае прототип функции будет бесполезным
static int add(int x, int y)
{
    return x + y;
}
```

Предварительные объявления функций не нуждаются в ключевом слове `extern`. Компилятор может определить сам (по типу функции) определяется ли вы функцию или пишете её прототип.

Файловая область видимости vs. Глобальная область видимости

Термины «файловая область видимости» и «глобальная область видимости», как правило, вызывают недоумение, и это отчасти объясняется их неофициальным использованием. В теории, в языке C++ все глобальные переменные имеют файловую область видимости. Однако, по факту, термин «файловая область видимости» чаще применяется к внутренним глобальным переменным, и «глобальная область видимости» — к внешним глобальным переменным.

Например, рассмотрим следующую программу:

global.cpp:

```
int g_x(5); // глобальная переменная, доступна в других файлах
```

main.cpp:

```
#include <iostream>

extern int g_x; // предварительное объявление g_x. Теперь g_x можно использовать в любом месте этого файла

int main()
{
    extern int g_x; // предварительное объявление g_x. Теперь g_x можно использовать только внутри main()

    g_x = 6;
    std::cout << g_x; // выведет значение 6

    return 0;
}
```

Если предварительное объявление находится вне блока, то оно применяется ко всему файлу. Если же внутри блока, то оно применяется только к нему.

Если переменная объявлена с помощью ключевого слова `static`, то получить доступ к ней с помощью предварительного объявления не получится. Например:

constants.cpp:

```
static const double g_gravity(9.8);
```

main.cpp:

```
#include <iostream>

extern const double g_gravity; // не работает g_gravity = constants.cpp, так как g_gravity является внутренней переменной

int main()
{
    std::cout << g_gravity; // выведет ошибку компилятора, так как переменная g_gravity не была определена для использования в этом файле
    return 0;
}
```

Обратите внимание, если вы хотите объявить неинициализированную неконстантную глобальную переменную, то не используйте ключевое слово `extern`, иначе C++ будет думать, что вы пытаетесь записать предварительное объявление.

Связи функций

Функции имеют такие же свойства связи, что и переменные. По умолчанию они имеют внешнюю связь, которую можно сменить на внутреннюю с помощью ключевого слова `static`:

```
// Функция add() определена в этом файле и может использоваться в других файлах
// Обратите внимание, что в этом случае прототип функции будет бесполезным
static int add(int x, int y)
{
    return x + y;
}
```

Предварительные объявления функций не нуждаются в ключевом слове `extern`. Компилятор может определить сам (по типу функции) определяется ли вы функцию или пишете её прототип.

Файловая область видимости vs. Глобальная область видимости

Термины «файловая область видимости» и «глобальная область видимости», как правило, вызывают недоумение, и это отчасти объясняется их неофициальным использованием. В теории, в языке C++ все глобальные переменные имеют файловую область видимости. Однако, по факту, термин «файловая область видимости» чаще применяется к внутренним глобальным переменным, и «глобальная область видимости» — к внешним глобальным переменным.

Например, рассмотрим следующую программу:

global.cpp:

```
int g_x(5); // глобальная переменная, доступна в других файлах
```

main.cpp:

```
#include <iostream>

extern int g_x; // предварительное объявление g_x. Теперь g_x можно использовать в любом месте этого файла

int main()
{
    extern int g_x; // предварительное объявление g_x. Теперь g_x можно использовать только внутри main()

    g_x = 6;
    std::cout << g_x; // выведет значение 6

    return 0;
}
```

Переменная `g_x` имеет файловую область видимости внутри global.cpp. Доступ к этой переменной вне файла global.cpp отсутствует. Обратите внимание, хотя это переменная и используется в main.cpp, case main.cpp не видит ее, он видит только предварительное объявление `g_x` (которое также имеет файловую область видимости). Линкер отвечает за связывание определения `g_x` в global.cpp с использованием `g_x` в main.cpp.

Глобальные символьные константы

На уроке о **символьных константах**, мы определяли их следующим образом:

constants.h:

```
#ifnef CONSTANTS_H
#define CONSTANTS_H

// Глобальные или глобальные пространство имен для хранения констант
namespace Constants
{
    const double pi(3.14159);
    const double avogadro(6.02214e23);
    const double my_gravity(9.2);
    // ... другие константы
}

#endif
```

Хоть это просто и отлично подходит для небольших программ, но каждый раз, когда constants.h подключается в другой файл, каждая из этих переменных копируется в этот файл. Таким образом, если constants.h подключить в 20 различных файлов, то каждая из переменных продублируется 20 раз. Header guards не остановят это, так как они только предотвращают подключение заголовочного файла более одного раза в один файл. Дублирование переменных на самом деле не является проблемой (поскольку константы занимают не занимают много памяти), но изменение значения одной константы потребует перекомпиляции каждого файла, в котором она используется, что может привести к большим временным затратам в более крупных проектах.

Избежать этой проблемы можно, превратив эти константы в константные глобальные переменные, и изменив **заголовочный файл** только для хранения предварительных объявлений переменных. Например:

constants.cpp:

```
#include <iostream>

namespace Constants
{
    extern const double pi(3.14159);
    extern const double avogadro(6.02214e23);
    extern const double my_gravity(9.2);
}
```

constants.h:

```
#ifnef CONSTANTS_H
#define CONSTANTS_H

// Глобальные или глобальные пространство имен для хранения констант
namespace Constants
{
    // Только предварительные объявления
    extern const double pi;
    extern const double avogadro;
    extern const double my_gravity;
}

#endif
```

Их использование в коде остается неизменным:

```
#include "constants.h"

// ...
double circumference = 2 * radius * Constants::pi;
// ...
```

Теперь определение символьных констант выполняется только один раз (в constants.cpp). Любые изменения, сделанные в constants.cpp, потребуют перекомпиляции только одного этого файла.

Но есть и обратная сторона медали: такие константы больше не будут считаться константами типа compile-time и, поэтому, не смогут использоваться где-либо, где потребуется константа такого типа.

Поскольку глобальные символьные константы должны находиться в отдельном пространстве имен и быть доступными только для чтения, то использовать префикс `g_` уже не обязательно.

Предостережение о (неконстантных) глобальных переменных

У начинающих программистов часто возникает соблазн использовать просто множество глобальных переменных, поскольку с ними легко работать, особенно когда задействовано много функций. Тем не менее, этого следует избегать. Почему? Об этом мы поговорим на следующем уроке.

Заключение

Подытожим вышесказанное:

- Глобальные переменные имеют глобальную область видимости и могут использоваться в любом месте программы. Подобно функциям, вы должны использовать предварительные объявления (с ключевым словом `extern`), чтобы использовать глобальную переменную, определенную в другом файле.
- По умолчанию, глобальные неконстантные переменные имеют внешнюю связь. Вы можете использовать ключевое слово `static`, чтобы сделать их внутренними.
- По умолчанию, глобальные константные переменные имеют внутреннюю связь. Вы можете использовать ключевое слово `extern`, чтобы сделать их внешними.
- Используйте префикс `g_` для идентификации ваших неконстантных глобальных переменных.

Тест

В чём разницы между областью видимости, продолжительностью жизни и связью переменных? Какие типы продолжительности жизни, области видимости и связи имеют глобальные переменные?

Ответ

- Область видимости определяет, где переменная доступна для использования. Продолжительность жизни определяет, где переменная создается и где уничтожается. Связь определяет, может ли переменная использоваться в другом файле или нет.
- Глобальные переменные имеют глобальную область видимости (или «файловую область видимости»), что означает, что они доступны из точки объявления до конца файла, в котором объявлены.
- Глобальные переменные имеют статическую продолжительность жизни, что означает, что они создаются при запуске программы и уничтожаются при её завершении.
- Глобальные переменные могут иметь либо внутреннюю, либо внешнюю связь (это можно изменить через использование ключевых слов `static` и `extern`, соответственно).